



ITI · PYTHON FRAMEWORK WITH FASTAPI TRACK

Python Framework with FastAPI

Eng. Fatma Elraey

Lab 1 — Build a JSON-Only Documents REST API

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lab 1 of 5 — 1 Hour 48 Minutes
Focus	Hands-on construction of an in-memory multi-endpoint FastAPI service using GET, POST, PUT, DELETE, parameters, JSON, and HTTP status codes.
Evaluation	Lab Assignment — 100%

Before You Start

This lab continues the same Documents API used across the course. Keep each step working before moving on, and test every endpoint as you build it.

Today's Lab Plan

1. Create and run a FastAPI project
2. Build health and collection endpoints
3. Add path and query parameters
4. Implement POST, PUT, DELETE
5. Verify status codes with Swagger and curl

Install

```
pip install fastapi "uvicorn[standard]"
```

✓ QUICK CHECK — Before You Start

Q1. What command runs the development server?

Answer: `uvicorn main:app --reload`

Q2. What format should every application endpoint return?

Answer: `JSON`.

01 Project Setup & Health Endpoint

Goal: Create a minimal FastAPI application and confirm the server is reachable.

Steps

1. Create a folder named `fastapi_documents` and a file `main.py`.
2. Create `app = FastAPI(title="Documents API")`.
3. Add GET `/health` returning `{"status": "ok"}`.
4. Run Uvicorn and open `/docs`.

Starter / reference code

```
from fastapi import FastAPI

app = FastAPI(title="Documents API")

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

✓ CHECKPOINT — Exercise 1

Q1. What should GET `/health` return?

Answer: HTTP 200 with JSON `{"status": "ok"}`.

02 Build the In-Memory Resource

Goal: Create an in-memory documents collection and expose list/read endpoints.

Steps

1. Create a dictionary keyed by integer document id.
2. Implement GET `/documents`.
3. Implement GET `/documents/{document_id}`.
4. Return 404 with HTTPException when the id does not exist.

Starter / reference code

```
from fastapi import HTTPException

documents = {
    1: {"id": 1, "title": "FastAPI Notes", "content": "REST basics"}
}

@app.get("/documents")
def list_documents():
    return {"items": list(documents.values())}

@app.get("/documents/{document_id}")
def get_document(document_id: int):
    item = documents.get(document_id)
    if item is None:
        raise HTTPException(status_code=404, detail="Document not found")
    return item
```

✓ CHECKPOINT — Exercise 2

Q1. What should `/documents/999` return?

Answer: 404 Not Found.

03 POST — Create Documents

Goal: Accept a simple JSON body and return 201 Created.

Steps

1. Create POST /documents.
2. Accept payload: dict for this first lab.
3. Generate the next integer id.
4. Store the record and return it with status 201.

Starter / reference code

```
from fastapi import status

@app.post("/documents", status_code=status.HTTP_201_CREATED)
def create_document(payload: dict):
    new_id = max(documents.keys(), default=0) + 1
    item = {"id": new_id, **payload}
    documents[new_id] = item
    return item
```

✓ CHECKPOINT — Exercise 3

Q1. Why use 201 instead of 200?

Answer: *The request created a new resource.*

04 PUT & DELETE

Goal: Complete the CRUD-style lifecycle.

Steps

1. Implement PUT /documents/{document_id}.
2. Return 404 for an unknown id.
3. Implement DELETE /documents/{document_id}.
4. Return a clear JSON confirmation or 204 if you choose an empty response.

Starter / reference code

```
@app.put("/documents/{document_id}")
def update_document(document_id: int, payload: dict):
    if document_id not in documents:
        raise HTTPException(status_code=404, detail="Document not found")
    documents[document_id] = {"id": document_id, **payload}
    return documents[document_id]

@app.delete("/documents/{document_id}")
def delete_document(document_id: int):
    if documents.pop(document_id, None) is None:
        raise HTTPException(status_code=404, detail="Document not found")
    return {"deleted": document_id}
```

✓ CHECKPOINT — Exercise 4

Q1. What method updates/replaces a resource in this lab?

Answer: *PUT.*

05 Query Parameters & Client Testing

Goal: Add optional filtering and test the API from outside Python.

Steps

1. Add q and limit query parameters to GET /documents.
2. Open Swagger UI and execute every endpoint.
3. Repeat at least GET, POST and DELETE with curl.
4. Record the status code for one successful and one failed request.

Starter / reference code

```
from fastapi import Query

@app.get("/search")
def search_documents(
    q: str | None = Query(default=None),
    limit: int = Query(default=20, ge=1, le=100),
):
    items = list(documents.values())
    if q:
        items = [x for x in items if q.lower() in x.get("title", "").lower()]
    return {"items": items[:limit]}
```

✓ CHECKPOINT — Exercise 5

Q1. What is the difference between /documents/1 and ?limit=1?

Answer: *The first identifies one resource; the second modifies a collection request.*

06**Submission Checklist**

- main.py runs with Uvicorn.
- GET /health returns 200 JSON.
- GET /documents and GET /documents/{id} work.
- POST returns 201.
- PUT and DELETE handle missing ids with 404.
- At least one query parameter is implemented.
- All endpoints tested in Swagger UI; at least three tested with curl.